CLOUD COMPUTING

Name - Sourav Yadav
Roll No - 23010104 3
Github - https://github.com/sourav-ydv/cloud-computing-mini-project

This project took me through the complete lifecycle of deploying a ML model as a cloud service. I learned sequence-level knowledge distillation concretely. I ran Phi-3.5 (3.8B parameters) through Ollama to label 3000 commands with structured JSON, cleaned the outputs (2856 of 3000 kept after removing invalid JSON, wrong keys, bad intents, bad slots and duplicates), and fine-tuned Pythia-70M (70M parameters) on 1500 of them. My trained student received 40% full-frame accuracy against the teacher's 46% — 87% of the teacher's performance from a model roughly 54 times smaller.

I learned to use Pytorch, Hugging face transformers, SeaweedFS object storage (S3-compatible), FastAPI, Docker, and Kubernetes (K3d) together as one working system, not as isolated exercises.

Challenges and How I Overcame them

The hardest problem I hit was a single training step taking 300 secs instead of under 1 second. I isolated it by timing the forward pass separately from the backward pass and found the backward pass alone was the bottleneck. I traced this to using Pytorch 2.15 (the latest release at the time) together with the latest transformers, which together triggered a severe performance regression. Pinning both libraries to the versions specified in my own training/config.yaml fixed it completely - the same training step dropped to about 3 seconds.

A second major issue appeared only once I deployed the service to Kubernetes and ran load tests: under concurrent requests, pods entered a crash-restart loop and 99% of requests failed. I found that my /parse endpoint was declared async def but called the model's generate() method synchronously inside it, which blocked FastAPI's single event loop during each few-second generation. Health-check requests queued up behind it, timed out, and Kubernetes killed pods it believed were responsible. I fixed this by moving the blocking calls into a background thread with asyncio.to_thread() which let health checks keep responding while generation ran. After the fix, pod restarts dropped to zero across every subsequent load tests.

I also had networking trouble; kubectl port-forward to SeaweedFS repeatedly died, especially after a WiFi network change, interrupting a 281MB model transfer mid-stream. I switched SeaweedFS to a Kubernetes NodePort Service instead, which gave a stable, direct connection with no further drops.

Labelling 3000 commands with the teacher model took hours on CPU, so I added checkpointing to my worker script (saving progress every 25 records) so an interruption would never lose more than a few mins of work.

Observations, Findings, and Improvements

The biggest finding: deploying a model is a different skill from training one. My model itself worked correctly from early on; nearly all my later debugging time went into making it serve reliably under concurrent load - health probes, resource limits, and blocking vs non-blocking request handling mattered as much as model accuracy.

I also confirmed the CI quality gate actually works, not just that it exists: I deliberately pointed it at untrained baseline model and

watched it correctly fail (JSON validity near 0%, far under the 95% threshold), then restored the real trained model and watched the same pipeline pass cleanly.

If I started over, I would run a load test against the service immediately after writing it, before deploying to kubernetes - this would have caught the blocking-handler bug in minutes locally instead of rather after a confusing cluster of pod crashes. I would also pin dependency versions from the very first pip install, rather than discovering the version-mismatch slowdown through hours of debugging.

This project taught me that most of real engineering is debugging - reading stack traces carefully, isolating variables one at a time, and not assuming the first plausible explanation is the right one.